

Concurrent Programming TDA384/DIT392

22 August 2024

Exam supervisors: N. Piterman (piterman@chalmers.se, 031 772 6053)
and G. Schneider (gersch@chalmers.se, 031 772 6073)

(Exam set by N. Piterman and G. Schneider, based on the course given in
Aug-Oct 2023 and Jan-Mar 2024)

Material permitted during the exam (hjälpmedel):

Two textbooks; four sheets of A4 paper with notes (single or double-sided);
English dictionary (no smart phones allowed).

Grading: You can score a maximum of 70 points. Exam grades are: between 28–41 (3), between 42–55 (4), 56 or more (5).

Passing the course requires passing the exam and passing the labs. The overall grade for the course is determined as follows: between 40–59 (3), between 60–79 (4), 80 or more (5).

The exam **results** will be available in Ladok within 15 *working* days after the exam's date.

Instructions and rules:

- Please write your answers clearly and legibly: unnecessarily complicated solutions will lose points, and answers that cannot be read will receive no points!
- Justify your answers, and clearly state any assumptions that your solutions may depend on for correctness.
- Answer each question on a new page. Glance through the whole paper first; five questions, numbered Q1 through Q5. Do not spend more time on any question or part than justified by the points it carries.
- Be precise. In your answers, try to use the programming notation and syntax used in the questions. You can also use pseudo-code, *provided* the meaning is precise and clear. If need be, explain your notation.

Q1 (15p). In the rest of this question we assume an implementation of semaphores that does not allow to go beyond the capacity of the semaphore. In this implementation, the capacity is different from the initialisation, that is, when you create a new semaphore, you need to give both the capacity and the initial value of the semaphore. A call to `up()` will *block* when the value of the internal counter of the semaphore is equal to the capacity (i.e., the thread will wait till the semaphore's counter is strictly less than the capacity, acting in the same way as `down()` does when the counter is zero). Note that, however, if the initial value of the semaphore's internal counter is higher than the capacity, you can still call `down()`. When you define a semaphore you will need to provide both the initial value and the capacity. The down operation (calling `sem.down()`) works as usual.

For instance, if you want to define a semaphore `sem` with capacity 5 and initial value 2, you will declare it as follows:

```
sem = new semaphore(2)[capacity=5]
```

In the example, the internal counter of `sem` is set originally to 2, so it is possible to make three consecutive calls to `sem.up()`, but it will *block* if a thread tries to call `sem.up()` again (assuming no other thread called `sem.down()` before).

In what follows you will get 5 questions concerning locks and semaphores. You need to justify your answer in each case (an answer without a justification will not be granted full points).

1 **(2p)** The pseudocode of the program in Fig. 1 shows the interaction between two threads operating on a shared variable `count`. We assume that there is no other thread using the semaphore. For this particular example the behaviour of the program does not change if we define the capacity of the semaphore to be 1. True or false? Justify.

2 **(2p)** Let us consider again the pseudocode of the program shown in Fig. 1 (as before, we assume that there is no other thread using the semaphore). Let us assume we now declare a lock. In that program, and under the above assumptions, we can safely replace the semaphore by a lock by **only** performing the following replacements: each `sem.up()` is replaced by `lock.unlock()` and each `sem.down()` by `lock.lock()`. Is that true or false? (By *safely* we mean that the behaviour of the program is preserved when you do the replacement.)

If you answer *true*, then explain why the replacement preserves the original behaviour. If you answer *false*, explain why this is

int count; Semaphore sem = new Semaphore(1)[capacity=2];	
thread t	thread u
int a,b; 1 sem.down(); 2 a = count - 1; 3 b = a * 2; 4 count = a + b + 1; 5 sem.up();	int c; 6 sem.down(); 7 c = count + 1; 8 count = c - 1; 9 sem.up();

Figure 1: Q1(1-3): Pseudocode of program `someCounting`.

int count; Semaphore sem = new Semaphore(1)[capacity=2];	
thread t	thread u
int b; 1 sem.up(); 2 count = count + 1; 3 b = 2; 4 count = count + b; 5 sem.down();	int c; 6 sem.up(); 7 c = count + 1; 8 count = c - 1; 9 sem.down();

Figure 2: Q1(4-5): Pseudocode of program `someCounting2`.

not the case and give examples of wrong behaviours occurring because of the replacement (that highlights why the replacement is not correct).

- 3 (3p) Let us now consider the very same program shown in Fig. 1, but let us assume that there might be other threads running in parallel with threads `t` and `u` and that they may both call `sem.up()` and `sem.down()`. Under such assumption, is it true that the program satisfies the following properties: i) mutual exclusion; ii) deadlock freedom

If you answer *true*, then explain why this is the case. If you answer *false*, explain which properties are not satisfied and why (give an argument and at least one example that shows a violation of the property).

- 4 Let us now consider the pseudocode of the program shown in Fig. 2, and let us assume that there is no other thread using the semaphore. State whether the following assertions are true or false (justify your answer in both cases):
- i. **(2p)** The program satisfies: i) deadlock freedom; ii) mutual exclusion.
 - ii. **(2p)** Let us assume we declare a lock. We replace the semaphore by a lock by **only** performing the following replacements: each `sem.up()` is replaced by `lock.unlock()` and each `sem.down()` by `lock.lock()`. Is it true that the behavior of the program does not change?
- 5 Let us now consider again the program shown in Fig. 2, but let us assume that there might be other threads running in parallel with threads `t` and `u` and that may call both `sem.up()` and `sem.down()`. State whether the following assertions are true or false (justify your answer in both cases):
- i. **(2p)** The program satisfies: i) deadlock freedom; ii) mutual exclusion.
 - ii. **(2p)** Let us still assume that other threads may use the semaphore, but that now we have declared a lock which may be used by all threads. We replace the semaphore by a lock in the codes of threads `t` and `u` by **only** performing the following replacements: each `sem.up()` is replaced by `lock.lock()` and each `sem.down()` by `lock.unlock()`. Is it true that the behaviour of the program in Fig. 2 is preserved?

Q2 (14p). Malin is presenting her new painting collection and has invited some friends and relevant people from the artistic scene to an opening exhibition. Guests to the exhibition can arrive at any time without previous notice, but they are required to say goodbye to Malin before leaving (that is, they are not to leave unless Malin is available to greet them goodbye). Malin can go in and out of the exhibition at any time. We model this behaviour in Java pseudo-code using a monitor class, as explained below.

- Malin and her guests correspond to concurrent threads — one thread per person.
- Synchronisation occurs through an instance of monitor class Exhibit:

monitor class Exhibit implements IExhibit

The interface **IExhibit** is defined in Fig. 3:

- Malin and the guests share an instance of the class Exhibit. Malin keeps calling methods **enterMalin()** and **exitMalin()**, and the guests continuously call **arrive()** and **leave()** (one thread **Guest i** for each guest i ($1 \leq i \leq n$)) (see Fig. 4).
- Methods **enterMalin()**, **exitMalin()**, and **arrive()** are non-blocking; method **leave()** is blocking if Malin is currently **not** at the exhibition, and terminates as soon as Malin enters the party again.

The objective is to provide an implementation of a *monitor class* **Exhibit** with interface **IExhibit**, whose behaviour follows the above description. You should use the Java pseudo-code for monitors that we used in our lectures, or any similar notation that describes monitors and whose semantics is clear. NOTE: Assume a *signal and continue* signalling discipline.

(Part a). (3p)

Declare the *condition variables* and other *private attributes* that you need in class Exhibit. For each of them, explain them with a concise comment.

(Part b). (4p)

Write the implementation of methods **arrive()** and **leave()**.

(Part c). (4p)

Write the implementation of methods **enterMalin()** and **exitMalin()**.

(Part d). (3p)

Would you have to change your implementation if you were to use monitors following the *signal and wait* signalling discipline instead of *signal and continue*? Justify your answer.

```

interface IExhibit
{
    void enterMalin();    // Malin is available at the exhibition
    void exitMalin();    // Malin leaves the exhibition (momentarily)

    void arrive(int guest); // a guest arrives at the exhibition
    void leave(int guest);  // a guest leaves the exhibition
}

```

Figure 3: Q2: Pseudocode of interface `IExhibit`.

		<code>IExhibit exhibit = new Exhibit();</code>	
		<code>malin</code>	<code>guest i</code>
1	<code>while (true) {</code>		<code>while (true) {</code> 7
2	<i>// Stay some time:</i>		<i>// Arrive at exhibit.</i> 8
3	<code>exhibit.enterMalin();</code>		<i>// (i is guest's ID):</i> 9
4	<i>// Leave some time:</i>		<code>exhibit.arrive(i);</code> 10
5	<code>exhibit.exitMalin();</code>		<i>// Leave the exhibit.</i> 11
6	<code>}</code>		<i>// (when possible):</i> 12
			<code>exhibit.leave(i);</code> 13
			<code>}</code> 14

Figure 4: Q2: Pseudocode of threads of program `Exhibit`.

Q3 (15p). In this question, we give three implementations of a leader election protocol. From time to time threads try to become leaders by calling the *attempt* function. Independently, other threads call the *follow* function and get the thread ID of the leader. An implementation should satisfy the following. If someone is the leader for a certain interval of time then during that time no other thread can become a leader (mutual exclusion). A call to *follow* should return a leader value that is true to the time of the return (i.e., the leader or 0). Analyze which of the following implementations indeed creates such a leader election protocol. For each implementation, state which of the required properties hold (and explain why) and which do not hold (and explain why/give an example execution).

Here are the variables (and their types) that used by the three implementations. All three use the same constructor.

```
private Semaphore s = new Semaphore(1);
private Lock l = new ReentrantLock();
private AtomicLong expiry = new AtomicLong();

private long leader = 0;

public Leader() {
    expiry.set(System.currentTimeMillis());
}
```

(Part a). (5p) Here is an implementation attempt using a lock. Notice that the atomic long is used here as a normal long with only get and set and not with compare and set.

```
public void attempt1(long interval) {
    l.lock();
    try {
        if (expiry.get() >= System.currentTimeMillis())
            return;

        leader = Thread.currentThread().getId();
        expiry.set(System.currentTimeMillis() + interval);
    } finally {
        l.unlock();
    }
}

public long follow1() {
    if (expiry < System.currentTimeMillis()) {
        return leader;
    }
    return 0;
}
```

(Part b). (5p)

Here is an implementation attempt using an atomic long. Notice the use of compare and set unlike the previous implementation.

```
public void attempt2(long interval) {
    long current = expiry.get();
    if (current < System.currentTimeMillis()) {
        if (expiry.compareAndSet(current, System.currentTimeMillis() + interval)) {
            leader = Thread.currentThread().getId();
        }
    }
}

public long follow2() {
    if (expiry.get() < System.currentTimeMillis()) {
        return leader;
    }
    return 0;
}
```

(Part c). (5p)

Here is an implementation using a semaphore.

```
public void attempt3(long interval) {
    long current = expiry.get();
    if (current < System.currentTimeMillis()) {
        s.release();
    }
    if (s.tryAcquire()) {
        leader = Thread.currentThread().getId();
        expiry.set(System.currentTimeMillis() + interval);
    }
}

public long follow3() {
    if (s.tryAcquire()) {
        s.release();
        return 0;
    } else {
        return leader;
    }
}
```


Q4 (14p). In this question you are asked to modify the Readers-Writers board in Erlang that was taught in class to a Leader-Follower board.

Here is the original code presented in class:

```
% get read access to `Board`
begin_read(Board) ->
    Ref = make_ref(),
    Board ! {begin_read, self(), Ref},
    receive
    | {ok_to_read, Ref} -> ok_to_read
    end.

% release read access to `Board`
end_read(Board) ->
    Ref = make_ref(),
    Board ! {end_read, self(), Ref}.

% get write access to `Board`
begin_write(Board) ->
    Ref = make_ref(),
    Board ! {begin_write, self(), Ref},
    receive
    | {ok_to_write, Ref} -> ok_to_write
    end.

% release write access to `Board`
end_write(Board) ->
    Ref = make_ref(),
    Board ! {end_write, self(), Ref}.

% board with no readers and no writers
empty_board() ->
    receive
    | % serve read request
      {begin_read, From, Ref} ->
        From ! {ok_to_read, Ref}, % notify reader
        readers_board(1);         % board has one reader
    | % serve write request synchronously
      {begin_write, From, Ref} ->
        From ! {ok_to_write, Ref}, % notify writer
        receive                   % wait for writer to finish
        | {end_write, _From, _Ref} ->
          empty_board()           % board is empty again
        end
    end.

% board with no readers (and no writers)
readers_board(0) -> empty_board();
```

```

% board with `Readers' active readers (and no writers)
readers_board(Readers) ->
  receive
    % serve write request
    {begin_write, From, Ref} ->
      % wait until all `Readers' have finished
      [receive {end_read, _From, _Ref} -> end_read end
       || _ <- lists:seq(1, Readers)],
      From ! {ok_to_write, Ref}, % notify writer
      receive % wait for writer to finish
        {end_write, _From, _Ref} -> empty_board()
      end; % board is empty again
    % serve read request
    {begin_read, From, Ref} ->
      From ! {ok_to_read, Ref}, % notify reader
      readers_board(Readers+1); % board has one more reader
    % serve end read
    {end_read, _From, _Ref} ->
      readers_board(Readers-1) % board has one less reader
  end.

```

A Leader-Follower board works slightly differently. There can be only one leader inside the board at any given time. However, followers need to have a leader in order to be inside the board. The number of followers that can be in the board at a given time (when there is a leader in) is unbounded and they can go in and out freely (as long as there is a leader inside). Once the leader wants to go out, it needs to make sure that all followers have left (and it is OK to stop new ones from coming in) and only then leave the board, which then waits for the next leader.

Your task is to modify the given implementation to that of Leader-Follower. Just like in the readers-writers board, you may assume that leaders and followers follow the engage-disengage protocol. For example, only followers who completed successfully a call to `begin_follow` would call `end_follow`, and similarly for leaders.

No points will be deducted for syntax issues. Make sure that indentation and intention is clear.

(Part a). (1p)

Implement `begin_follow` and `end_follow`, which should be counterparts of `begin_read` and `end_read`.

(Part b). (3p) Implement `begin_lead` and `end_lead`, which should be counterparts of `begin_write` and `end_write`.

(Part c). (5p) Implement `empty_board`.

(Part d). (5p) Implement `lead_board`.

It is allowed to design the board as one (or more than two) functions.
In such a solutions the 10 points will be allocated according to the
functionality supported.

Q5 (12p). In what follows you have 3 sub-questions (Parts a to e) concerning different topics seen in the course. Each part a multiple-choice question. The grading for each question is as follows:

- For a right answer you will get 3 points;
- If you do not answer the question, you will get 0 points;
- If you answer wrongly, you will get -1 (a negative point) for each wrong choice.

The total amount of points will be done summing all the points for each part. So, if you answer correctly Part a, incorrectly Part b, and do not answer any of the rest, you will get 2 points (3 points for Part a minus 1 point for Part b, and 0 for the rest). Note that no negative points for the whole question will be given (the minimum number of points you may get for the whole question is 0). That is, if you do answer wrongly in each part, you will not get -5 but 0.

NOTE: For each sub-question, there is exactly one option that is correct.

(Part a). (3p)

To find the total number of combinations of size R from a set of size N , where R is less than or equal to N , we use the following *combination* formula:

$$\frac{N!}{R! (N - R)!}$$

Fig. 5 shows an Erlang parallel algorithm to compute the above formula.

What is the maximal number of processes running in parallel when executing `parallelcomb` with parameters $N = 4$ and $R = 3$?

- a) 1
- b) 3
- c) 4
- d) $\ln(7)$ ($\ln$ is the natural logarithm)
- e) $8 (N + R + (N - R))$
- f) $13 ((N * R) + (N - R))$

(Part b). (3p)

According to Amdahl's law, if the fraction p of a program can be parallelised, then, the maximum speedup that can be achieved by n processes is $\frac{1}{(1-p) + \frac{p}{n}}$.

We want to apply Amdahl's law to the Erlang program from sub-question a) above (program shown in Fig. 5), to calculate the speedup

```

-module(parallel_combinations).
-export([parallelComb/2, start/0]).

parallelComb(N, R) when N >= R, R >= 0 ->
    Parent = self(),
    spawn(fun() -> Parent ! {n_fac, factorial(N)} end),
    spawn(fun() -> Parent ! {r_fac, factorial(R)} end),
    spawn(fun() -> Parent ! {nr_fac, factorial(N-R)} end),

    N_Fac = receive {n_fac, Value} -> Value end,
    R_Fac = receive {r_fac, Value} -> Value end,
    NR_Fac = receive {nr_fac, Value} -> Value end,

    N_Fac div (R_Fac * NR_Fac).

factorial(0) -> 1;
factorial(N) when N > 0 -> N * factorial(N - 1).

```

Figure 5: Q5 - Part a: Erlang program `parallelComb`.

produced by this parallel version with respect to a purely sequential version of the program.

Which one of the following assertions is true?

- a) There is no speedup at all of the parallel version w.r.t. the purely sequential version.
- b) There is a speedup of 3, given that there are three spawned processes.
- c) The parallel version can run more than 10 times faster than the purely sequential one.
- d) The question is meaningless since Amdahl's law is used to make a prediction about the maximum performance improvement that can be expected from parallelising a program, and not to compare two versions of the same program.
- e) None of the above.

(Part c). (3p)

The monitor class below implements a strong semaphore:

```

monitor class StrongSemaphore implements Semaphore {
    int count;
    Condition isPositive = new Condition(); // is count > 0?

    public void down() {
        while (count > 0)
            count = count - 1;
        isPositive.wait();
    }

    public void up() {
        if (isPositive.isEmpty())
            count = count + 1;
        else isPositive.signal();
    }
}

```

- a) Yes. Assuming that there are no spurious wakeups and interruptions.
- b) Only under signalling policy signal and continue.
- c) Only under singnalling policy singal and wait.
- d) No.

(Part d). (3p)

The monitor class below implements a barrier.

```

public class BarrierMonitor {
    private int expect;
    private int current;

    public BarrierMonitor(int n) {
        this.expect = n;
        this.current = 0;
    }

    public synchronized void await() throws InterruptedException {
        current++;
        if (current < expect) {
            wait();
        }
        else {
            current = 0;
            notifyAll();
        }
    }
}

```

- a) Yes. Assuming that there are no spurious wakeups and interruptions.
- b) Only under signalling policy signal and continue.

- c) Only under signalling policy signal and wait.
- d) No.